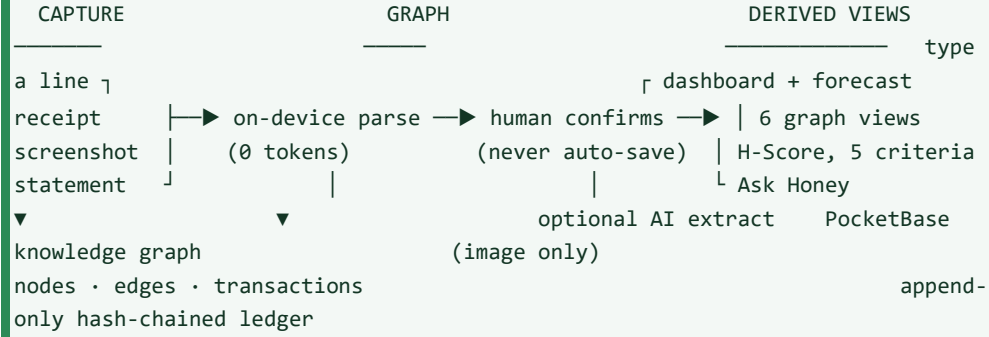


Technical Architecture Document - HoneyMoney

Team JUST50 · 27 August 2026 · Live at [honeymoney.app](#)

1. The system in one view



Every household is one graph. **Money is not a table of rows** — income flows along allocation edges into buckets, and buckets pay vendors — which is what lets the app see structurally that one bucket's velocity will push a shared goal late, before it does.

2. Layers

Layer	What runs	Design constraint it satisfies
1. App	Next.js (App Router) PWA, Tailwind, six locales, multi-currency	Phone-first, installable, nothing to download. Server-rendered, so a midrange Android does no heavy work.
2. Data	PocketBase — a single self-hosted binary: database, auth, migrations	Self-hostable by a household or an employer. Collections are superuseronly; no browser touches them.
3. Capture	Tesseract OCR and our own parser and classifier, in the browser	Zero tokens, zero transmission, by default. Works offline. No bank connection, ever.
4. AI	Gemini Flash, Groq, or local Ollama — switchable per deployment	Optional and replaceable. Per-call cost ledger. No lock-in, and no model in the critical path.
5. Delivery	Cloudflare Tunnel, Pages, R2	A public surface that stays up when the origin does not; backups leave as ciphertext.

3. The knowledge graph

Nodes

- income_source — carries monthly_amount ALLOCATES_PCT / ALLOCATES_FIXED — the plan bucket — tier 1 Must-paid, 2
- Savings, 3 Spendings FUNDS — a source funding a goal
- vendor , goal , obligation , asset

Edges

- SPENT_AT — a bucket paid a vendor
- Closed by valid_to , never deleted: history stays legible

Three invariants hold the model together. (1) *Income is read only from declared sources*, never inferred from a credit — a refund cannot become a salary. (2) *A transfer is not a spend*: money moved into Savings nets to zero at household level, so putting money away never renders as spending it. (3) *Void, never delete*: the ledger is append-only and hash-chained, so a correction is evidence rather than erasure.

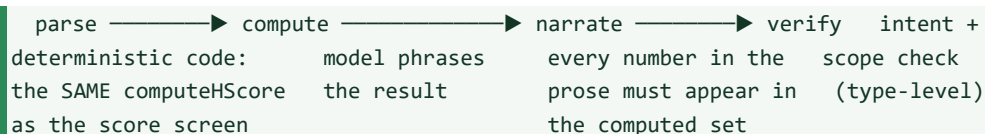
4. Derived views — computed, never stored twice

- **Allocation walk** — a depth-guarded traversal from each income source, so five income streams and nine buckets resolve with no schema change.
- **Projection** — month-to-date pace against each bucket's allocation: *on track*, *at risk*, *over budget*, or *unfunded*.
- **H-Score** — five weighted criteria on tuned curves: savings rate (30), essential burden (25), debt service (20), emergency buffer (20), personal cap (5); banded Building / Steady / Strong / Thriving, with hysteresis so a score cannot flicker.

5. Capture — the on-device path is the default, not the fallback

A typed line is parsed and classified in the same tick as the keystroke: amount, vendor, date, currency, and one of six categories, by keyword tables that a judge can read and a household can correct. **No network, no token, no account.** The identical module runs the landing page's try-it box, so what a visitor tries is the product, not a mock-up of it. A photographed receipt is OCR'd in the browser and itemised there. Only if a household switches AI on, and only for image extraction, does an image reach a model — and **nothing is ever saved without a human confirming it.**

6. Ask Honey — the model may not do arithmetic



Stage 2 emits the only figures allowed to appear. Stage 3 may choose words and emphasis; each number in its prose is checked against that set, and **one unrecognised figure discards the whole answer** for a deterministic template. That template is the floor, not the fallback: with no AI key at all, the answer is the same answer, less warmly phrased. On a cloud engine the payload is slot names, an intent and a locale — **no figures, no labels, no free text** — restored locally after the check. Below 8 records or 14 days, Honey declines to project.

7. Security and privacy architecture

Concern	Mechanism
Isolation	Every query is tenant-scoped; collections are superuser-only, reached through routes that authenticate the caller. A household's own AI key is encrypted (AES-256-GCM) before storage and never returned to a browser.
Partner privacy	Tier-3 buckets are personal by default. One visibility filter redacts the record list, the graph and Ask Honey — the same function in the same query, so a chat box cannot summarise what a list hides.
AI consent	Off until switched on, per purpose, append-only and withdrawable, enforced at the call site rather than the settings screen. Payloads carry a data class; class 2 prefers a local engine.
User backups	Zero-knowledge. Sealed in the browser — gzip, PBKDF2-HMAC-SHA256 at 600,000 rounds, AES-256-GCM, envelope parameters bound as AAD — with a passphrase we never receive. We store ciphertext and hold no key. The server refuses a payload that is not encrypted.
Operator backups	Nightly, encrypted before upload, held off-site, and restore-tested rather than assumed.
Integrity	Append-only hash-chained ledger; a record is voided, never rewritten.

The limit of the claim, stated rather than implied: the live database is not encrypted to the operator, because the server computes an H-Score over it. HoneyMoney is not an end-to-end encrypted ledger; the backup is the artefact that is sealed.

8. Deployment

The Next.js server runs on hardware the team controls, published through a named Cloudflare Tunnel, with a Pages snapshot serving the public surface when the origin is not up. PocketBase runs on a Singapore host with settings encryption. The whole stack is **one binary plus one Node app** — which is what makes self-hosting a real option rather than a slide, at near-zero infrastructure cost.

9. How each claim is verified

Command	What it proves against real code and a real database
<code>check:ask</code>	The model cannot introduce a number; scope holds at the type level; a partner cannot extract private records through the chat box.
<code>check:vault</code>	No plaintext survives into a backup; a wrong passphrase, a downgraded KDF and a flipped bit all refuse to open.
<code>check:capture</code>	What you type is what gets stored; a bare credit never becomes a salary.
<code>check:tally</code>	The graph and the dashboard agree, household by household, and every ringgit of income is routed.
<code>check:attribution</code>	Transfers net to zero and privacy holds inside the query, not just in the component.

10. Known limits

OCR is imperfect; every extraction is confirmed before filing. Restoring a household *from* a sealed backup is not built — opening one returns the data, but recreating a household needs id remapping the ledger has a view on. The seal runs in JavaScript we serve, so independent verification of the delivered bundle is a real gap, listed as one. PDPA 2010 alignment is not certified compliance; no third-party audit yet.